

Guia Completo de Utilizador do ATC

Guia Completo de Utilizador do Advanced Trigonometry Calculator

Versao: ATC 2.1.7 Idioma: Portugues de Portugal Autor: Renato Alexandre dos Santos Freitas

Este e o guia de utilizador expandido do Advanced Trigonometry Calculator (ATC). Foi construido a partir do guia online existente, da documentacao 2.1.7 atual e da cobertura automatizada de regressao deste repositório.

1. O que e o ATC

O ATC e uma aplicacao matematica gratuita, open source e de linha de comandos para Windows. Foi desenhada para trabalho matematico local rapido atraves de comandos e expressoes textuais.

Casos tipicos:

- calculos cientificos rapidos;
- trigonometria e trigonometria inversa;
- numeros complexos;
- matrizes;
- estatistica e probabilidade;
- funcoes DSP;
- simplificacao polinomial;
- resolucao de equacoes e sistemas;
- variaveis e expressoes reutilizaveis;
- modulos interativos guiados;
- processamento de ficheiros TXT.

2. O que o ATC nao e

O ATC foca-se deliberadamente em computacao matematica pratica orientada por comandos, em vez de tentar ser um CAS universal.

- O ATC nao e um CAS geral completo como Mathematica, Maple, SageMath ou SymPy.

- O ATC nao foi desenhado para provar teoremas matematicos arbitrarios.

- O ATC nao substitui ferramentas profissionais de validacao especificas de engenharia.

- O ATC foca-se em calculo numerico, fluxos por comandos, uso educativo, funcionalidades hibridas simbolicas/numericas implementadas e execucao local rapida.

- Existe comportamento simbolico apenas onde esta explicitamente suportado e testado.

3. Bases de input

As expressoes sao escritas diretamente no prompt.

```
2+3*4
sqrt(9)
sin(pi/2)
```

`log(100)`

Outputs estaveis:

```
#0=14
#1=3
#2=1
#3=2
```

O ATC usa `_` como marcador negativo em muitos inputs e outputs documentados:

`_6+2`

Resultado esperado:

```
#0=-4
```

Notacao cientifica usa E maiusculo:

```
1E3
1E-3
```

4. Prompt interativo

O ATC 2.1.7 inclui editor de linha:

- Tab completa comandos, funcoes matematicas, aliases e funcoes do utilizador;
- Tab repetido alterna entre completamentos possiveis;
- Up e Down navegam no historico;
- Left, Right, Home, End, Delete e Backspace editam a linha atual.

5. Modo angular

Use:

```
mode
```

Opcoes:

```
radian -> 1
degree -> 2
gradian -> 3
```

Exemplos:

```
sin(pi/2)
sin(30)
```

Use `sin(pi/2)` em radianos e `sin(30)` em graus.

6. Precisao

O ATC 2.1.7 suporta:

- `double`
- Boost `mp_float`

Alternancia persistente:

```
higherprecision(1)
higherprecision(0)
```

A definicao e aplicada apos reiniciar o ATC.

Comandos uteis:

```
dp15dppi
dp50dppi
dp50dpe
dp50dmaxprecpi
```

Exemplos validados:

```
dp50dppi
3.14159265358979323846264338327950288419716939937511
```

```
dp50dpe
2.71828182845904523536028747135266249775724709369996
```

maxprec permite usar mais precisao numa expressao sem alterar o modo persistente.

7. Constantes e respostas anteriores

Constantes comuns:

```
pi
e
i
INF
_INF
true
false
```

Resultados anteriores:

```
#0
#1
```

8. Aritmetica

Exemplos:

```
2+3*4
(2+3)*4
8/4
2^10
5!
abs(_7)
100rest(3)
100quotient(3)
rtD4D(16)
```

Resultados esperados:

```
2+3*4      -> 14
(2+3)*4    -> 20
2^10       -> 1024
5!         -> 120
abs(_7)    -> 7
rtD4D(16)  -> 2
```

9. Trigonometria

```
sin(pi/2)
cos(0)
tan(pi/4)
sec(0)
cosec(pi/2)
cotan(pi/4)
asin(1)
acos(1)
atan(1)
acosec(1)
asec(1)
acotan(1)
```

Em modo grau:

```
mode
sin(30)
```

Resultado esperado:

```
#n=0.5
```

10. Funcoes hiperbolicas

```
sinh(0)
cosh(0)
```

```
tanh(0)
sech(0)
cosech(1)
cotanh(1)
asinh(0)
acosh(1)
atanh(0)
asech(1)
acosech(1)
acotanh(2)
```

11. Logaritmos

```
log(100)
ln(e)
logb2b(8)
```

Resultados esperados:

```
log(100)  -> 2
ln(e)     -> 1
logb2b(8) -> 3
```

12. Numeros complexos

```
2i
3(2i)
sin(30+30i)
asin(sin(30+30i))
```

O modo angular influencia funcoes trigonometricas com argumentos complexos.

13. Multiplicacao automatica

```
2pi
pi2
2e
2(3+4)
(1+1)(2+3)
2sin(pi/2)
sin(pi/2)2
2i
3(2i)
```

Com variaveis:

```
x=4
2x
x(4)
(4)x
xpi
pix
xe
xsin(pi/2)
```

Quando existe uma variavel com nome exato, o ATC deve preferir essa variavel em vez de dividir o nome em produtos implicitos.

14. Variaveis

```
data=5
data+1
energy=10
energy+1
```

Comandos uteis:

```
see variables
renamed variables
eliminate variables
see results
eliminate results
```

15. Matrizes

Criar matriz:

```
create matrix(foo\2\2\3)
```

Helpers:

```
min(foo)
max(foo)
avg(foo)
linsnum(foo)
colsnum(foo)
getlins(foo)
getcols(foo)
```

Operacoes:

```
foo+foo
foo-foo
foo*foo
foo^2
foo^T
foo^R
foo/2
2*foo
```

16. Estatistica e probabilidade

```
min(3\_1\2)
max(3\_1\2)
avg(2\4\6)
```

Resultados:

```
min(3\_1\2) -> -1
max(3\_1\2) -> 3
avg(2\4\6) -> 4
```

Helpers gaussianos/probabilidade:

```
gerror(0)
gerrorc(0)
gerrorinv(0)
gerrorcinv(1)
qfunc(0)
qfuncinv(0.5)
```

Modulo guiado:

```
statistics calculations
```

17. DSP

```
sinc(0)
sinc(1)
fft(1\0\0\0)
fft(1\2\3)
ifft(1\0\0\0)
ifft(1\1\1\1)
```

18. Polinomios

Simplificacao:

```
simplify polynomial(x)
simplify polynomial(x+1)
simplify polynomial(x+x+1)
simplify polynomial(x^2-5*x+6)
simplify polynomial((x-2)*(x-3))
simplify polynomial((x+1)*(x+1))
simplify polynomial((x-1)^2)
simplify polynomial((x+2)*(x-2))
simplify polynomial((x+2i)*(x-2i))
simplify polynomial(((x-5)(x+2))/(x-5))
```

Raizes para polinomio:

```
roots to polynomial(2\3)
roots to polynomial(2\7\12)
roots to polynomial(6.5i\_6.5i)
roots to polynomial(1+2i\1-2i)
roots to polynomial(0\1\2)
```

Exemplo:

```
roots to polynomial(2\3)
(1+0i)x^2+(_5+0i)x^1+(6+0i)
```

19. Resolucao de equacoes

Quadraticas:

```
solve quadratic equation(1\_5\6)
solve quadratic equation(1\_2\1)
solve quadratic equation(1\2\5)
```

Sistemas:

```
solve equations system(1\1\5;1\_1\1)
```

Equacoes polinomiais:

```
solve equation(x+2)
solve equation(x^2-5*x+6)
solve equation(x^7-12)
solve equation(1\2\3\4\5)
solve equation((x-1)(x-2))
solve equation(((x-5)(x+2))/(x-5))
solve equation(x^25-100)
```

Exemplos:

```
solve equation(x+2)
x1=-2
```

```
solve equation(x^2-5*x+6)
x1=3
x2=2
```

20. Solver

```
solver(sin(x)-0.5)
solver(0.5-sin(x))
solver(cos(x)-cos(60))
solver(tan(x)-tan(45))
solver(asin(sin(30))-x)
solver(qfunc(x)-qfunc(0.34233))
solver(x^2-12x-9)
solver(x+2)
solver(((x-5)(x+2))/(x-5))
solver((x-e+pii)(x-e-pii))
```

O modo angular e importante nos exemplos trigonometricos.

21. Estudo de funcoes

```
function study(x+1)
function study(x^2)
function study(x^2-4)
function study(_x^2)
function study(1/(x-1))
function study((x^2-1)/(x-1))
function study((1-x^2)/(x^2-4))
function study(x/(x^2-9))
function study((x+1)/(x^2+1))
```

O estudo pode apresentar dominio, zeros, assintotas, simetrias, sinal, monotonia, concavidade e contradominio quando suportado.

22. Grafico

```
graph settings
graph(x)
graph(x;_2\2\1)
```

O grafico e renderizado na consola. A validacao visual completa continua a ser mais adequada manualmente.

23. Modulos guiados

```
triangles rectangles solver
arithmetic matrix solver
financial calculations
geometry calculations
statistics calculations
physics calculations
unit conversions
microeconomics calculations
```

Estes modulos pedem valores e opcoes ao utilizador.

24. Receitas por area

Financas:

```
financial calculations
100*15/100
```

Geometria:

```
geometry calculations
```

Fisica:

```
physics calculations
```

Conversoes:

```
unit conversions
```

Microeconomia:

```
microeconomics calculations
```

25. Tempo

```
time
calendar(2026)
day of week(y2026m6d20)
day of week(d20m6y2026)
time difference calculations
actual time response
```

Ferramentas como stopwatch, timer, big timer, clock e big clock devem ser usadas manualmente quando envolvem tempo real ou janelas abertas.

26. Ordenacao

```
ascending order(3\1\2)
descending order(3\1\2)
ascending order(_3\1\_2\0)
descending order(_3\1\_2\0)
ascii order
inverse ascii order
```

27. TXT processing e command bridge

```
predefine txt
solve txt
solve txt(name)
open txt("C:\Temp\file.txt")
to solve
enable txt detector
eliminate strings
atc from cmd
atc over cmd
```

```
auto solve txt
```

Fluxo tipico:

1. Criar um TXT com um comando ATC por linha. 2. Executar `predefine txt` e indicar o caminho. 3. Executar `solve txt`. 4. O ATC cria um ficheiro `_answers.txt`.

Exemplo de TXT:

```
2+2
sqrt(9)
solver(x-2)
solve equation((x-1)(x-2))
simplify polynomial((x+1)(x-1))
2++
2+3
```

O ATC deve continuar apos uma linha invalida e reportar o erro dessa linha.

28. Scripting

O guia online documenta helpers como:

```
print(...)
get(...)
sprintf(...)
while
for
if
else
switch
case
cls()
break
return
replace
replace by index
count occurrences
delete x occurrences
is contained
calc
is equal
get positive value
is to write
is variable
is contained variable
is contained by index
strlen
```

Use scripting para fluxos repetiveis e teste scripts pequenos antes de os usar em calculos maiores.

29. Funcoes do utilizador

As funcoes do utilizador vivem na pasta `User functions`. O autocomplete pode sugerir funcoes como `atc_<nome>()`.

30. Settings e ambiente

```
current settings
verbose resolution
verbose resolution(1)
verbose resolution(0)
numerical systems
si prefixes
actual time response
dimensions
window
about
auto adjust window
enable atc intro
disable atc intro
clean history
clean
exit
```

31. Ficheiros e pastas

O ATC gere pastas como:

- pasta de dados ATC;
- Scripts examples;
- Source code;
- Strings;
- To solve;
- User functions.

Alguns comandos abrem Explorer, Notepad ou visualizadores externos.

32. Exportacao

Alguns comandos perguntam:

```
Export result? (Yes -> 1 \ No -> 0)
```

Use 1 para exportar ou 0 para ignorar.

33. Verbose resolution

```
verbose resolution(1)  
verbose resolution(0)
```

O modo verbose ajuda a aprender e depurar, mas o uso normal e mais limpo com verbose desligado.

34. Perfis de utilizacao

Engenharia:

```
solve equation(x^2-5*x+6)  
create matrix(foo\2\2\3)  
foo^2  
unit conversions  
physics calculations
```

Educacao:

```
2+3*4  
sin(pi/2)  
mode  
geometry calculations  
function study(x^2)
```

DSP:

```
sinc(0)  
fft(1\0\0\0)  
ifft(1\1\1\1)  
sin(pi/2)+cos(0)
```

Estatistica:

```
avg(2\4\6)  
min(3\_1\2)  
max(3\_1\2)  
statistics calculations  
qfunc(0)
```

Financas:

```
financial calculations  
100*15/100
```

35. Resolucao de problemas

Se uma expressao falhar:

- verificar parenteses;
- verificar modo angular;
- confirmar se deve usar _ para negativo;
- confirmar se o comando e interativo;
- testar partes menores da expressao;
- usar `verbose resolution(1);`
- confirmar se a funcionalidade esta documentada e coberta por testes.

36. Documentacao e testes

```
docs/Testing.md
tests/ATC_AUTOMATED_TEST_CASES.md
tests/ATC_USER_GUIDE_COVERAGE.md
```

Resultado atual validado:

```
Summary: 359 passed, 0 failed
```

Este guia deve evoluir com os testes e com o comportamento documentado do ATC 2.1.7.